

WRS Final Submission

Mariusz Matyja

hkp680

1 Data (Week 1)

The dataset used in this project, “Musical Instruments” dataset from the Amazon Review, contains 14000 entries, each representing a user rating a musical instrument. Data points consists of the following: ‘user_id’, ‘item_id’, ‘rating’, and ‘timestamp’. The data is originally split into a test set, and a training set being 3000 rows, and 11000 rows in size. The id values are literals, and both rating and timestamp values are numerical values, where rating are in {1,2,3,4,5}. From this point onward, for ease of use, item id(s), and user id(s) will be used interchangeably with ‘iid(s)’ and ‘uid(s)’ shortcuts.

1.0.1 Data Cleaning

The item, and user id values were scanned for invalid values. The ratings were checked if all are in the predefined range (1-5). All of the data-points were valid in both splits. As the next step, duplicate rows were removed, and only the most recent entries, based on the timestamp values, were kept. This resulted in a significant data loss, especially in case of the test split. There were 1178 duplicates in the test part, and 1087 in the training part. Next, the user, and item ids in both splits were compared, revealing 9 items occurring in the test part, but not in the training set. The records containing the ids were removed as well. Lastly the intersection between the training records and test records was calculated resulting in an empty set, as expected.

1.0.2 Final Data Overview

Table 1 represents the final number of entries in both data splits: before and after the cleaning procedure. Table 2 shows the number of unique iids and uids in both splits. Table 3 shows the percentage distribution of rating values in the training split. Figure 1 shows the frequency of item ratings. Figure 2 presents the distribution of the

ratings given by users. One can observe, that both plots have ‘long tails’ meaning, only a small subset of users/items gives/receives significantly larger amount of rating than the majority.

Split name/Stage	Train	Test
Initially	11000	3000
Cleaned	9913	1728

Table 1: Number of entries in both splits before and after the cleaning process.

Split name/id type	Test	Train
unique uids	427	800
unique iids	404	500

Table 2: Number of unique item ids(iids), and unique user ids(uids) in both splits.

1.0.3 Data Insights

If there was a rating of all items for each user in the training data, the set would consist of approx. $800 \times 500 = 400\,000$ entries. The available training data (a bit over 9900), equal merely approx 2.44% of the full ranking matrix. Thus, one can conclude the data is sparse. This can show to be troublesome when learning recommendation algorithms. Furthermore, the test set after the cleaning procedure is only 18% of size of the train split. Moreover, Table 3 shows a clear tendency to rate high among the users in the training split. One could deduce, the skewness can lead to poor performance in machine learning methods when applied to the data. The percentage of 4’s and 5’s is approx. 88,7% of all training points, which could point to a binary approach to the rating values e.g. : {1,2,3} = dislike, and {4,5} = like. The long tails of in the frequency plots in figures 1 and 2 also indicate, that naive algorithm can easily fail when learning on the data. It is because given the initial sparsity of

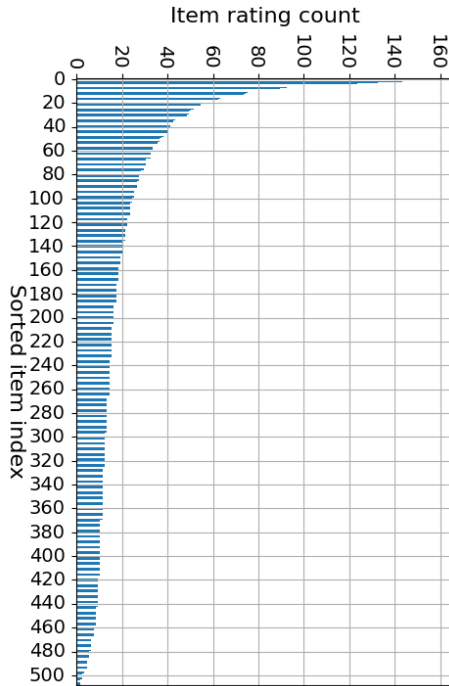


Figure 1: Frequency of rating pr item. The x axis shows amount of times an item was rated. The y axis represent an items position when sorted by the amount of rating received

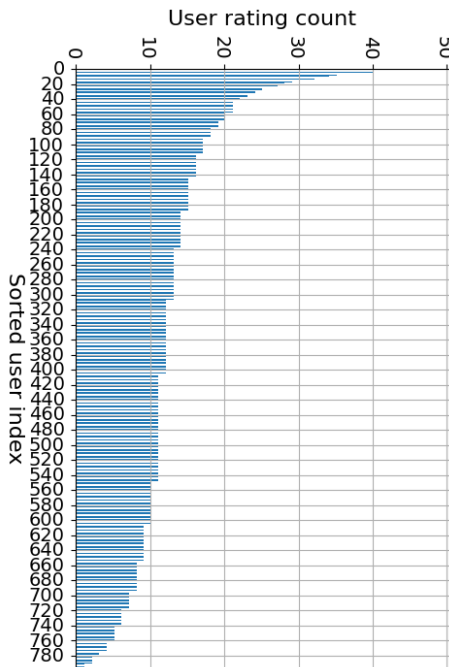


Figure 2: Frequency of rating pr user. The x axis shows amount of times a user rated an item. The y axis represent a user's position when sorted by the amount of rating given

the data, the long tails indicate, that a significant amount of records are associated with few users, and few items. Lastly, popular items seems to be rated highly, as can be observed in Table 4

Rating value	1	2	3	4	5
Percentage	2,3%	2,5%	6,4%	17%	71,7%

Table 3: The percentage of particular value ratings of all ratings in the training split

item id	avrg rating	total ratings
B0BSGM6CQ9	4.69	158
B0BPJ4Q6FJ	4.79	156
B09857JRP2	4.77	143
B0BCK6L7S5	4.3	132
B08R5GM6YB	4.5	123

Table 4: Top 5 items with highest number of ratings from the training split.

2 Simple Models

2.1 Models and Hyperparameters (Week 2)

In this section, three models will be produced, to predict the unobserved rating for users in the test set. TopPop, KNN-means, and SVD algorithms will be optimized, and trained on the available data. Existing implementations of KNN and SVD algorithms were used, in particular the 'Surprise' Python library.¹ Only choice of parameters, and important functions will be commented on.

2.1.1 TopPop

TopPop is a simple algorithm recommending the items with the highest number of rating exceeding a particular threshold (e.g. 3). Given the simplicity of the algorithm the implementation details will not be provided here. The threshold was chosen to be 3. The decision was based on previously discussed rating values distribution in the training split.

2.1.2 KNN

KNNWithMeans was chosen as the algorithm from KNN family. This particular variant takes into consideration mean rating of each user, when finding neighbors, and predicting unobserved ratings. To find the optimal parameters GridSearchCV class from the same library was utilized. The process consisted of a 5-fold cross-validation over all

¹<https://surprise.readthedocs.io/en/stable/>

combination of chosen parameters. These were $k \in \{2, 5, 10, 15, 20\}$, as well as two similarity calculation methods: cosine and Pearson. To measure the error during the parameter search RMSE was chosen. The choice of RMSE over MSE was motivated by RMSE being more meaningful in case of rating values. The squared error metric penalize outliers heavier than other similar metrics, therefore it can be a good fit for data, where a minority of users/items shift the rating distribution significantly, which is a case in this project, as earlier observed. Another advantage as, that the metric was already implemented in the named library thus saving workload.

The grid search yielded the following parameters: $k = 20$, and cosine similarity measure. All results of the grid search can be observed in Table 5.

K neighbours/ sim. measure	2	5	10	15	20
pearson	0.91	0.93	0.91	0.91	0.91
cosine	0.98	0.91	0.91	0.91	0.91

Table 5: The table shows mean rmse score values for all 5 folds obtained through hyperparameter grid search for the KNN algorithm. The considered parameters were similarity measure (pearson coeff. and cosine sim.), and the amount of negihbours (2,5,10,15 and 20). The values are round up to 2 decimal places.

2.1.3 SVD

epochs/factors	5	10	15	20
2	0.86	0.85	0.85	0.85
3	0.86	0.85	0.85	0.85
5	0.86	0.85	0.85	0.85
10	0.86	0.85	0.85	0.85

Table 6: The table shows mean rmse score values for all 5 folds obtained through hyperparameter grid search for the SVD algorithm. The considered parameters were number of latent factors (2,3,5,10), and the amount of epoches (5,10,15 and 20). The values are round up to 2 decimal places.

SVD class from the Surprise library was utilized to train a latent factor model. The chosen version is the most ordinary one. Once again GridSearchCV, and 5-fold cross validation was used to find optimal parameters. The chosen metric was once again RMSE for the previously mentioned reason, as well as for ease of initial comparison of the two algorithms. The tested params. were:

latent factors $\in \{2, 5, 10, 15, 20, 30\}$, and number of epoches $\in \{5, 10, 15, 20, 30\}$. The grid search yielded the following parameters: 2 latent factors, and 20 epoches. All results of the grid search can be observed in Table 6.

2.2 Evaluation (Week 3)

2.2.1 RMSE of Test data prediction

Algorithm:	KNN	SVD
RMSE:	1.06	0.97

Table 7: The table shows rmse score for test split predictions for both KNN and SVD algorithms trained on the whole test split, and using optimal parameters yielded by the grid searches.

During grid search for optimal hyperparameters for both the KNN and SVD models RMSE was used to choose the best combinations. Unfortunately, even for the optimal parameters the error values were high. These can be observed in Table 7. The RMSE metric, as mentioned earlier, penalizes outliers stronger. This can be advantageous, but not necessarily. In a setting where it is significantly more important to predict accurate on average, and rarely occurring significant errors in predictions are not dangerous, RMSE would not be a good fit. However, when it is crucial not to introduce outliers in predictions, and small deviations from ground truth on average are not important RMSE is a fine measurement. One could argue, that recommended systems are the second case, since e.g. in case of rating between 1 and 5 it can be not as important to predict a rating, that differs from truth by 0.2, as to not recommend a completely irrelevant item to a user.

2.2.2 Metrics for Top 10 Measurement

HitRate measures how many users receive at least one relevant recommendation in the top k. It does not take in consideration how many of the recommended items are relevant, nor at which position they are presented. Thus the score would be 1.0(perfect), if a model never recommended an item in the top 5 out of 10 recommendations, but always recommended at least 1 relevant item out of the 10. It is an obvious shortcoming of the metric. Precision takes into account how big fraction of the recommended items were relevant. This provides more information than HitRate, but this metric does not incorporate how high in the recommendation were the relevant items.

MRR incorporates the position of the first relevant item among recommendation, thus providing information not observable using the above described two measurements. On the other hand the number of relevant items in the set of recommendations is not measured by MRR.

Finally, MAP metric takes into account both the number of relevant recommendations, as well as their placement in the top. One of the shortcoming of all of the three metrics, is that there is no account for the actual number of observed relevant items, as in case of e.g. Recall. The Coverage measures how large percentage of all items are on average recommended to users.

2.2.3 Top 10 Recommendations

The top 10 recommendation were predicted for all users in the test split, and all items in the training split. Recommendations of the three models were evaluated with use of HitRate, Precision, MAP, MRR, and Coverage metrics. Table 8 shows the results for the three algorithms.

Additional insights in the three algorithms performance can be gain by analyzing the impact of the long tail. Users were sorted by their activity (same as in Figure 2), then 20% of the top, and the bottom were chosen. The same matrices as before were computed, but only for recommendation of top and bottom 20%. The results can be observed in Tables 9 and 10.

In Table 8 one can easily notice, that TopPop scores highest ratings, except for coverage. This follows from the very nature of TopPop i.e. recommending the same items to all users. KNN and SVD perform significantly worse than TopPop, and very similar to each other. The exception is KNN's coverage score, that is 5 times as large as SVD's. The trend is similar in Table 10 for the 20% least active users. In this case, the TopPop scores very high HitRate of approx. 0.46. However for the 20% most active users the trend is opposite i.e. KNN scores highest HitRate.

The low performance of CF methods on the whole train split, and the least active 20% of users, can either mean, that the predictions are semi-random due to inefficient amount/quality of data, or because users simply prefer the most popular items. One could assume, that in the beginning users are interested in the most popular items, and first later, after purchasing and rating them they would move to less rated items. This assumption seems to be supported by the results for the 20%

most active users. Considering the sparsity of both the training and the test data one could further assume the available data represent mostly new users, and/or users with low activity. Summing up, the low average user activity, and the sparsity of the data could explain the poor performance of the CF algorithms.

There is another argument supporting the observations. While calculating the metrics for the whole training split two users were inspected. One with RR score higher than 0.05, and one with score below the threshold. The user with higher RR rated 10 items, and out of it's 10 neighbors(KNN based): 7 rated only 1 item, 2 rated 2 items each, and one rated 3 items. Whereas the lower RR user rated only 7 items, and only one of it's neighbors rated more than 1 item. This once again underlines how important role the sparsity of the available data plays in CF algorithms performance.

	KNN	SVD	TopPop
HitRate	0.098	0.096	0.255
Precision@10	0.01	0.01	0.034
MAP@10	0.029	0.025	0.136
MRR@10	0.029	0.024	0.117
Coverage	0.519	0.112	0.02

Table 8: Five metrics computed for all test users with the three recommendation algorithms trained on the whole train split. The values are round up to 3 decimal places. The bold values are highest values in the respective row.

	KNN	SVD	TopPop
HitRate	0.155	0.121	0.138
Precision@10	0.016	0.012	0.016
MAP@10	0.041	0.028	0.076
MRR@10	0.041	0.028	0.072
Coverage	0.206	0.094	0.02

Table 9: Five metrics computed for all test users with the three recommendation algorithms trained on 20% of most active users form the train split. The values are round up to 3 decimal places. The bold values are highest values in the respective row.

3 Text Representation (Week 4)

In this section additional data will be used, namely metadata containing a title, and a description of each in the training split. Both titles, and descriptions are of natural language, more precisely english. All of the available metadata will be first

	KNN	SVD	TopPop
HitRate	0.106	0.181	0.475
Precision@10	0.012	0.019	0.068
MAP@10	0.029	0.040	0.255
MRR@10	0.027	0.039	0.216
Coverage	0.43	0.067	0.02

Table 10: Five metrics computed for all test users with the three recommendation algorithms trained on 20% of least active users from the train split. The values are round up to 3 decimal places. The bold values are highest values in the respective row.

preprocessed, and then transformed into tfidf and the Google’s word2vec word embeddings. The choice to include all of the data was made based on the simple rule ”The more data the merrier”. First, after closer inspecting usefulness of the word embeddings the final decision of what part of the data to use will be made.

3.1 Text Preprocessing

The first step after tokenization of the metadata, English stopwords, and special characters were removed, since they usually do not carry high percentage of a sentence meaning. Next, the text tokens were lemmatized, to decrease the size of the vocabulary, while preserving as much meaning as possible. Also, Lemmatization was chosen over Stemming for the same reason. Upon closer inspection of the descriptions, and titles a decision of removing words not officially in english dictionary. This was done using one of the largest online english word corpora. The motivation to do so, followed from observing many proper names e.g. product names, especially in the titles. Lastly, due to still relatively large vocabulary as for the TF-IDF representation, words with document frequency under 2 were removed. There were two vocabularies created one for each of title and description parts. This was done, so it is possible to measure how useful can each of them be alone. From now on the vocabularies will be referred to as Dvoc, and Tvoc. Later, a combination of word embed. from both vocabularies will be introduced. Table 11 shows the change in the size of the title and description vocabularies after some of the preprocessing steps.

3.2 Word Embeddings

In order to implement content based recommender algorithms one needs to convert literal tokens into numerical values to represent items, and users. As

	Description	Title
Initially	8255	2156
Eng. words - stop words	2620	761
Doc. freq. >2	1351	296

Table 11: Size of the vocabulary of both title and description metadata parts. Each row shows result of a consecutive step.

mentioned earlier the chosen representations are: TF-IDF, and Word2Vec(or W2V) vectors. Moreover as stated before, two separate vocabularies were created: Dvoc and Tvoc. In order to compare vector representations across word embedding types, and vocabularies four distinct representations were created: DvocTfidf, TvocTfidf, DvocW2V, TvocW2V. Additionally to inspect combined impact of Dvoc and Tvoc representations DTvocW2V, and DTvocTfidf were created using the other four. Each DTvocW2V was created by summing and averaging respective DvocW2V and TvocW2V vectors. This could be done since the feature represent the same kind of information independently of used vocabulary. On the other hand, it is not a case with TF-IDF representation, therefore TvocTfidf were appended at the end of DvocTfidf vectors, and then to reduce the dimension Principal Component Analysis was utilized. The size of each embedding type was 300 features, except for TvocTfidf, and DvocTfidf, which sizes were 1351, and 296 features respectively(the size of the respective vocabularies).

3.3 Vector Similarities

The next step on the way to implement CB models, is to choose means of comparing vector representations. Similarity measure was chosen since it is well known, broadly used, and it’s range is positive, which improves ease of metrics computations.

After computing vector representations of all items, item-item similarity matrix was computed, which was used to draw first insights about the embedding types. Some items did not have either title or description in metadata file, which led to zero-vectors, and in consequence 0 similarity score.

As much as over 90% of all item-item similarities were non-zero for W2V vectors, and only around above 40% for Tf-IDF vectors. Also the W2V similarities are on avrg. higher than TF-IDF equivalents.

Table 12 shows chosen item-item similarities.

one can observe, that in case of pair 1 the Dvoc similarities for both TF-IDF and W2V are similar, but for pair 2 differ by 0.3. Also TvocTfidf sim. score is in both cases lower than the DvocTfidf. The scores of combined vocabularies are between the scores for the two vocabs. alone.

	DvocTfidf	TvocTfidf	DTvocTfidf
pair 1	0.91	0.61	0.67
pair 2	0.73	0.51	0.56
	DvocW2V	TvocW2V	DTvocW2V
pair 1	0.99	0.91	0.92
pair 2	0.89	0.82	0.83

Table 12: The table shows similarity score for two pairs of items, for each of the vector representation types.

From manual data discovery it became clear, that DTvocTfidf, and DTvocW2V don't necessarily provide additional information compared to separate vocabs. First top recommendation measurements supported the observation. Additionally, algorithms using Tvoc vectors perform constantly worse than Dvoc's. Thus, to avoid chaos in the paper from this point onward only the following will be used: DvocTfidf, DvocW2V.

4 Content-Based Recommendations (Week 5)

From manual data discovery it became clear, that DTvocTfidf, and DTvocW2V don't necessarily provide additional information compared to separate vocabs. First top 10 recommendation measurements supported the observation. Additionally, algorithms using Tvoc vectors perform constantly worse than Dvoc's. Thus, to avoid unnecessary chaos in this paper, from this point onward only the following will be used: DvocTfidf, DvocW2V, and TvocTfidf (to numerically prove the claim of Title data inferiority).

4.0.1 User profile representation

To be able to recommend items for user, the users need to be represented in the same space as items. Each user profile was created by taking an averaged sum of the vector representations of the items the user rated. Initially all rated items were used, where the items rated below 4 were subtracted from user representation. This however showed to have minimal to none impact. Therefore only ratings above 3 were used.

4.0.2 Top 10 Recommendations

In order to rank items by similarity for each single user a user-item similarity matrix was calculated for both DvocTfidf and DvocW2V. Then based on the user-item similarity 10 top recommendations were chosen. The initial results were as bad as uniform random recommendations. The average similarity between the users and the items they rated was inspected. It revealed, that users do not rate items most similar to their profiles. This makes sense considering, that user representation is an average of (mostly) multiple items, which can have little in common. An example could be, that a user, who bought and rated a pack of 6 guitar strings most probably would not buy the exact same strings again, but in a pack of 12. Such two items would most probably have almost identical title and description differing only in the amount of strings in the package, leading to high similarity score between the user and the second item. The same would apply to two guitars in two different colors.

To improve the recommendations an average similarity between each user and its rated items was calculated. Later when choosing recommendation for a user, instead of simply choosing highest similarity scores, a difference between the similarity score and the avg. rated items similarity was considered.

The results can be seen in Table 13. All three methods performed poorly. TvocTfidf scored lowest in all metrics. DvocTfidf and DvocW2V perform similarly, but DvocTfidf slightly better. When comparing with CF algorithms (Table 8) it can be observed, that there is no negative improvement in all metrics. The TopPop algorithm still scores highest in all except coverage.

	DvocTfidf	DvocW2V	TvocTfidf
HitRate	0.110	0.117	0.066
Precision@10	0.012	0.013	0.007
Map@10	0.032	0.034	0.013
Mrr@10	0.030	0.032	0.013
Coverage	0.077	0.041	0.132

Table 13: The five metrics for top 10 recommendation for all test users based on the three embedding vector types. Values are rounded up to 3 decimal places.

5 Hybrid Systems (Week 6)

In this section implementation of three hybrid recommendation systems will be described, and mea-

sured. Each system will incorporate both a CF model, and a content-based model. For this the KNN model was chosen in all three hybrids due to its high coverage score. The CB part of the hybrid models will be implemented for the two best performing embedding vector types: DvocTfidf, and DvocW2V.

5.1 Pipeline Model

The Pipelining strategy introduces output of one model as input to another one. The implemented solution uses user-item similarity values for the user item combinations present in the training split. The values are used instead of ratings to train a KNN model. This idea came from the assumption, that the skewed distribution of ratings in the available data makes the standalone KNN overshoot the predicted ranking scores, ending with almost only 5's. As can be observed in Table 14, the strategy did only worsen the quality of recommendations.

	DvocTfidf	DvocW2V
HitRate	0.098	0.070
Precision@10	0.010	0.008
Map@10	0.035	0.027
Mrr@10	0.035	0.025
Coverage	0.224	0.171

Table 14: The five metrics for top 10 recommendation for all test users based on pipeline strategy. Values are rounded up to 3 decimal places.

5.2 Parallel Model

Parallel combination of recommended systems re-ranks recommendation based on some aggregation of ranks of two separate models. The implemented model re-ranks KNN recommendations by multiplying them with user-averaged(mentioned before) user-item similarity scores. Once again the model did not achieve better results than non-hybrid solution. The scores can be seen in Table 15.

5.3 Switching Strategy

Model incorporating the switching uses recommendation ranking either from one or other separate model, based on some predefined condition. This implementation switches between KNN and Content-Based recommendation depending on how active the user is in the training split. If a user rated less than a threshold $C=7$ items, then KNN takes over, otherwise the CB model. Value of C was

	DvocTfidf	DvocW2V
HitRate	0.063	0.073
Precision@10	0.007	0.008
Map@10	0.025	0.032
Mrr@10	0.025	0.031
Coverage	0.556	0.527

Table 15: The five metrics for top 10 recommendation for all test users based on the parallel model. Values are rounded up to 3 decimal places.

chosen empirically. Once again the results did not improve, which can be observed in Table 16.

	DvocTfidf	DvocW2V
HitRate	0.063	0.061
Precision@10	0.007	0.007
Map@10	0.017	0.016
Mrr@10	0.015	0.015
Coverage	0.870	0.611

Table 16: The five metrics for top 10 recommendation for all test users based on the switching strategy. Values are rounded up to 3 decimal places.

Revisions:

Avoid using implementation specific details about the code (e.g. lines 25-26), rather explain what you did. State which duplicates you removed, e.g. the most or least recent?

Table 1: Clarify whether this is the training or test set. You should report how many users/items/interactions are in the cleaned sets in a table.

Put the stats stated at the start of section 1.0.2 in a table.

Present the optimal models as well as the RMSE (missing) of the gridsearch, ideally in a table.

The report is missing a table with the optimal hyperparameter values after the 5-fold training, and the best RMSE scores averaged over the 5 folds.

In section 2.2.1, put the scores at the beginning of the first paragraph in a table.

Coverage is missing from the top 10 recommendations evaluation. Missing: discussion of model performance with respect to the rank-based evaluation measures